

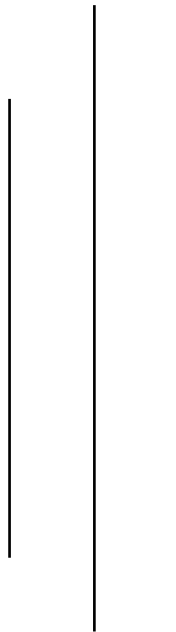
A Case Study

On

Personal Expense Tracker

Submitted partial fulfillment of the requirement of

Software Engineering (BIT 354CO)



Submitted By:

Samir Poudel (Roll No: 33)

Submitted To:

Ananta Bhadra Lamichhane

KANTIPUR CITY COLLEGE

Putalisadak, Kathmandu

August, 2026

Acknowledgement

I would like to express my sincere gratitude to our respected teacher, Mr. Ananta Bhadra Lamichhane, for his valuable guidance, support, and encouragement throughout the preparation of this case study for the Software Engineering subject. His teaching, knowledge, and suggestions helped us understand the concepts of software engineering and apply them practically while working on this case study.

I am truly thankful for the opportunity provided by him to explore real-world software engineering practices and enhance our analytical and problem-solving skills. I would also like to thank everyone who supported us directly or indirectly during the completion of this work. His dedication and continuous guidance have played an important role in making this case study successful.

Table of Contents

Chapter 1: Introduction	1
1.1 Background	1
1.2 Problem Statement:	1
1.3 Introduction	1
1.3.1 Project Objectives	1
1.3.2 System Scope	2
1.3.3 Project Scope	2
1.3.4 Nature of Anticipated Software	2
1.3.5 Essential Characteristics of Anticipated Software	2
1.4 Project Packaging	3
1.4.1 Work Breakdown Structure (WBS)	3
1.4.2 Deliverance and Milestones	3
1.4.3 Acceptance Criteria	4
1.4.4 Project Approach	4
1.5 Project Resourcing	5
1.5.1 Quality Requirements	5
1.5.2 Process Requirements	5
1.5.3 Method Requirements	6
1.5.4 Resource Requirements	7
Chapter 2: Planning and Management	9
2.1 Project Risks	9
2.1.1 Identified Risk	9
2.1.2 Mitigation Risk	10
2.1.3 Contingency Plan	10
2.1.4 Risk Monitoring Plan	11
2.2 Project Estimation and Schedule	12

2.2.1 Estimation Method Used.....	12
2.2.2 Project Estimation.....	12
2.2.3 Project Schedule.....	13
2.3 Configuration Management Plan	13
2.3.1 Change Management Plan	13
2.3.2 Version Management Plan	14
2.3.3 Release Management Plan	14
Chapter 3: System Engineering	15
3.1 Requirements	15
3.1.1 Domain Requirements	15
3.1.2 Functional Requirements	15
3.1.3 Non-Functional Requirements	16
3.1.4 Requirement Management Process.....	16
3.2 Design	18
3.2.1 System Architecture	18
3.2.2 High Level Designs.....	19
3.2.3 Data Structure Design	20
3.3 Testing.....	21
3.3.1 Test Justification	21
3.3.2 Test Plan.....	22
3.3.3 Test Design.....	23
Chapter 4: Monitoring & Control	26
4.1 Monitoring & Control	26
4.1.1 Time Monitoring	26
4.1.2 Cost Monitoring.....	26
4.1.3 Control Measures	27
4.2 Quality Assurance	28

4.2.1 QMS Selection	28
4.2.2 Quality Factors	28
4.2.3 Metrics for Process and Process Measurements	28
4.2.4 Quality Monitoring	30
Annex 1: Requirement Models	32

Chapter 1: Introduction

1.1 Background

Managing personal finances has become increasingly important in today's digital lifestyle. Individuals track daily expenses, income, and savings to maintain financial discipline. Traditional methods such as notebooks or simple spreadsheets are error-prone, time-consuming, and lack analytical features. A dedicated desktop application developed in Python can provide an efficient, offline, and user-friendly solution for recording, categorizing, and analyzing personal expenses.

1.2 Problem Statement:

Managing personal expenses effectively remains a major challenge for many individuals. Several limitations in existing methods and systems contribute to poor financial tracking and reduced control over spending. The major problems are:

Specific: Manual notebooks and spreadsheets fail to record daily income/expense transactions with proper categorization, leading to disorganized financial data.

Measurable: Over 70% of individuals using notebooks or spreadsheets report calculation errors or missing entries within a month of use.

Achievable: A lightweight, offline Python desktop application can be built within 8 weeks to automate recording, categorization, and reporting.

Relevant: Accurate expense tracking directly improves personal budgeting, savings, and financial decision-making.

Time-bound: The system must be capable of generating daily, monthly, and yearly reports instantly upon request, eliminating delays caused by manual calculation.

1.3 Introduction

1.3.1 Project Objectives

The objective of the project is:

Priority	Objective
1	Develop a user-friendly Python desktop application for recording daily income and expenses
2	Enable proper categorization of expenses and generation of summary reports
3	Ensure reliable local data storage
4	Create a simple, maintainable and easy-to-use system

1.3.2 System Scope

The system scope of the project is:

Priority	System Scope
1	Users can add, edit, and delete income and expense records
2	System supports predefined and custom expense categories
3	System generates monthly totals and category-wise summary reports
4	All data is stored locally on the user's computer

1.3.3 Project Scope

The project scope of the project is:

Priority	Project Scope
1	Requirements elicitation, analysis, and preparation of Software Requirements Specification
2	Design of system architecture, user interface, and database
3	Implementation of core modules using Python
4	Testing, documentation, and final delivery of the desktop application

1.3.4 Nature of Anticipated Software

The nature of the Anticipated Software is:

- Stand-alone desktop application that runs offline
- Interactive transaction-based system
- Custom-built software for personal expense management
- Local data storage and processing using SQLite.

1.3.5 Essential Characteristics of Anticipated Software

The essential characteristics of anticipated software are:

- **Usability:** Simple and intuitive graphical user interface
- **Reliability:** Accurate calculations and consistent data storage
- **Maintainability:** Modular and well-structured Python code
- **Portability:** Runs on Windows and Linux operating systems
- **Efficiency:** Fast response for data entry and report generation

1.4 Project Packaging

1.4.1 Work Breakdown Structure (WBS)

The work breakdown structure of the project is depicted in below diagram:

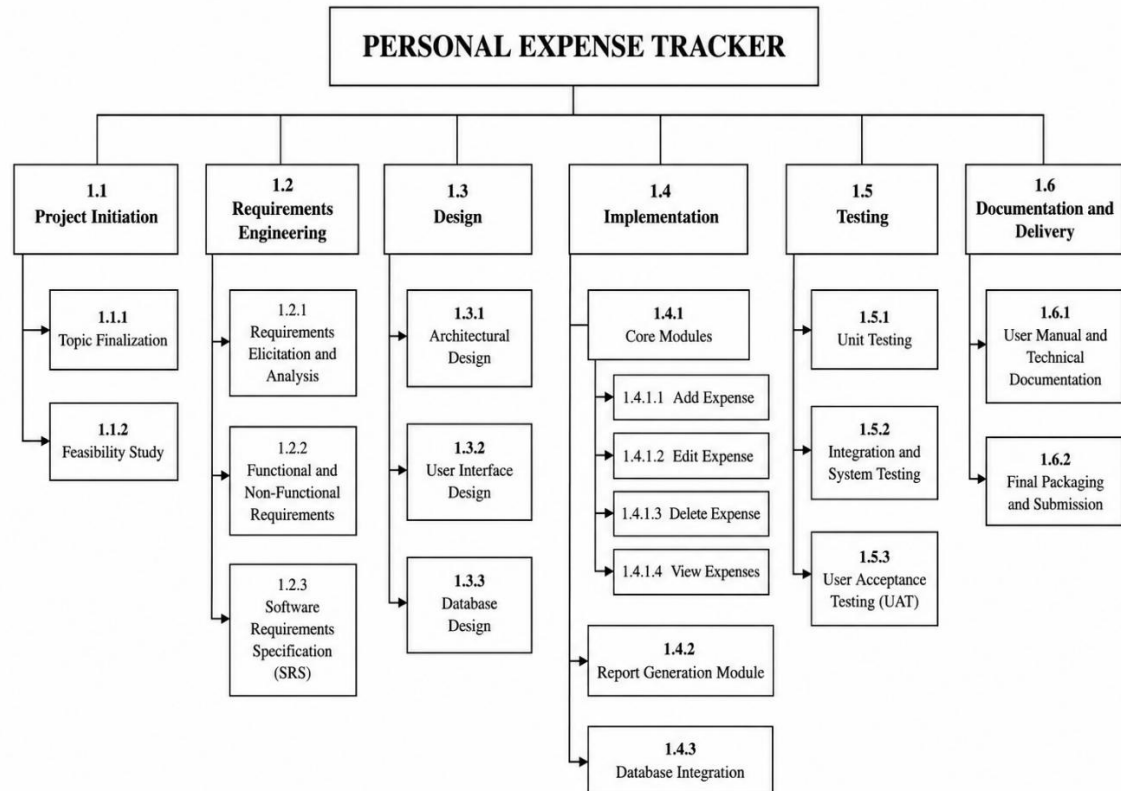


Fig 1.1 WBS Chart

1.4.2 Deliverance and Milestones

The list of deliverance and milestones are:

Sequence	Deliverable	Milestone (Completion Time)
1	Validated Project Topic and Project Plan	Week 1
2	Software Requirements Specification (SRS)	Week 2
3	System Design Document (UML, Database & UI Design)	Week 3
4	Working Prototype (Core Modules Implemented)	Week 5
5	Complete Application with Testing	Week 7
6	Final Documentation and Project Submission	Week 8

1.4.3 Acceptance Criteria

The list of acceptance criteria is:

S.N.	Acceptance Criteria
1	The application installs and launches successfully on the target operating system (Windows).
2	Users can add new expense records with details such as date, category, amount, and description.
3	Users can edit and delete existing expense records without data loss.
4	Users can view all recorded expenses in a clear and organized table.
5	Users can search and filter expenses by date, category, or keyword.
6	The application correctly calculates total expenses for a selected period.
7	Reports (daily, monthly, or yearly) are generated accurately.
8	All expense data is stored and retrieved correctly from the database.
9	Input validation prevents invalid or incomplete data from being saved.
10	The application performs without critical errors or crashes during normal operation.
11	The application passes unit testing, integration testing, and User Acceptance Testing (UAT).
12	User manual and technical documentation are completed and approved by the supervisor.

1.4.4 Project Approach

The Incremental Process Model was selected for the development of the Personal Expense Tracker because of the following reasons:

1. **Modular Development:** The application consists of separate modules such as Expense Management, Category Management, Reports, and Database Management. Each module can be developed independently.
2. **Early Working Software:** A functional version of the application is available after each increment, allowing progress to be evaluated throughout development.
3. **Continuous Testing :** Every completed module is tested before integrating it with the next module, helping to detect and fix defects early.
4. **Easy Maintenance:** Changes or improvements can be made to a specific module without affecting the entire application.

5. **Reduced Development Risk:** Developing the project in smaller increments minimizes the risk of major errors and makes debugging easier.
6. **Better Project Management:** The project can be planned, monitored, and completed in phases, making development more organized and manageable.
7. **Suitable for the Project:** Since the Personal Expense Tracker is a desktop-based application with clearly defined and independent functionalities, the Incremental Process Model provides an efficient and systematic development approach.
8. **Fits the project scenario:** With a single developer, an 8-week academic timeline, and clearly separable functionalities with stable requirements, incremental delivery provides a low-risk, well-managed path to completion — better suited here than models built for evolving requirements.

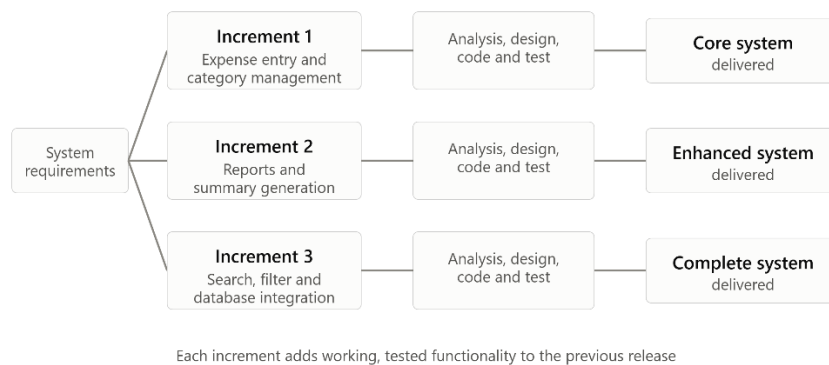


Fig 1.2: Incremental Model

1.5 Project Resourcing

1.5.1 Quality Requirements

The quality requirements the project aims to fulfil are:

- Correctness of calculations and data storage
- Reliability during normal use
- High usability through a simple interface
- Maintainable source code
- Completeness of core features

1.5.2 Process Requirements

The quality requirements are mapped to the Incremental Process Model as follows, to ensure each requirement is met throughout development:

S. N	Quality Requirement	Mapped Activity in Incremental Process	How Quality is Ensured
1.	Correctness	Testing phase of each increment	Every increment is tested individually before integration, catching calculation/logic errors early
2.	Reliability	Continuous validation across increments	Each new increment is validated against previously delivered increments to maintain stable behavior
3.	Usability	Design phase of each increment	UI is reviewed and refined incrementally based on feedback from early working versions
4.	Maintainability	Modular development structure	Each increment is built as an independent module, keeping code isolated and easy to modify
5.	Completeness	Incremental delivery sequence	Each increment adds a clearly defined feature set, ensuring no core functionality is missed by the final release

1.5.3 Method Requirements

Throughout the development of the Personal Expense Tracker, a set of key framework and umbrella activities shall be followed to ensure compliant and systematic process adoption. The **framework activities**, representing the core process steps carried out within every increment, are as follows:

- **Communication:** Requirements are clarified and validated with the project supervisor before the start of each increment to ensure accurate understanding of expected functionality.
- **Planning:** Tasks for each increment are scheduled and organized to maintain a structured development timeline.
- **Modeling:** The architecture, user interface, and database design relevant to each increment are prepared prior to implementation.
- **Construction:** Each module is coded and unit-tested to verify correctness before integration with the rest of the system.
- **Deployment:** The completed and tested increment is integrated into the overall system and delivered as a working version of the application.

In addition to the framework activities, the following **umbrella activities** are carried out continuously throughout the project to ensure that the development process remains compliant, controlled, and of consistent quality:

- **Project tracking and monitoring:** Progress is regularly monitored against the planned schedule to ensure timely completion of each increment.
- **Risk management:** Potential risks are identified early and mitigation measures are applied to minimize their impact on the project.
- **Software quality assurance:** Reviews and testing are conducted at each stage to ensure that the software meets the defined quality requirements.
- **Software configuration management:** Version control is maintained using Git to track changes and manage different versions of the source code.
- **Documentation management:** All project documentation, including design, testing, and user manuals, is prepared and maintained systematically throughout the development lifecycle.

1.5.4 Resource Requirements

The successful development of the Personal Expense Tracker requires the following technological, platform, environment, and human resources.

Technological Resources

- Programming Language: Python 3.x
- GUI Framework: Tkinter
- Database: SQLite
- Development Tool: VS Code / PyCharm
- Version Control: Git

Platform and Environment

- Operating System: Windows 10/11 or Linux
- Hardware: Standard desktop/laptop with minimum 4 GB RAM

Human Resources

- Project Developer: 1 (Student)
- Project Supervisor: 1
- End Users for feedback: 2–3

Chapter 2: Planning and Management

2.1 Project Risks

2.1.1 Identified Risk

Each identified risk is rated on Probability and Impact using a scale of 1–5 (1 = Very Low, 5 = Very High). The Risk Priority Number (RPN) is calculated as $RPN = Probability \times Impact$, and risks are sorted by RPN to determine which require the most urgent attention.

Risk ID	Risk Description	Category	Prob. (1-5)	Impact (1-5)	Risk Exposure	Priority
R1	Underestimation of development effort/time due to limited prior project experience	Schedule Risk	4	4	16	High
R2	Requirement changes after supervisor feedback (e.g., new report formats, added fields)	Product Risk	3	3	9	Medium
R3	Database design flaws causing incorrect calculations or data loss	Performance Risk	2	5	10	High
R4	Limited prior experience with Tkinter GUI framework slowing development	Technology Risk	3	3	9	Medium
R5	Single-developer illness/unavailability delaying milestones	Staff Risk	2	4	8	Medium
R6	Hardware failure or accidental file deletion causing loss of source code	Support Risk	2	5	10	High
R7	Underestimated testing time leading to schedule slip before submission	Schedule Risk	3	4	12	High

R8	Data validation gaps allowing invalid entries (negative amounts, wrong dates)	Cost Risk	3	2	6	Low
----	---	-----------	---	---	---	-----

2.1.2 Mitigation Risk

To reduce the likelihood or severity of each identified risk before it occurs, the following preventive actions have been planned and will be applied throughout development.

Risk ID	Mitigation Strategy
R1	Break work into small, testable increments (per WBS); track progress weekly against the 8-week schedule; build in a 3–5-day buffer before each milestone
R2	Freeze core requirements after SRS approval (Week 2); route any new request through a documented Change Request before implementation
R3	Design and normalize the database schema early (Week 3) and validate it with sample data before coding CRUD modules
R4	Complete a short Tkinter tutorial/prototype (login/dashboard mock) before starting core module development
R5	Maintain a personal task buffer and keep the supervisor informed of progress so deadlines can be renegotiated if needed
R6	Maintain regular Git commits and push to a remote repository (GitHub) daily; keep a local backup copy
R7	Allocate a dedicated testing phase (Week 6–7) in the schedule rather than testing only at the end
R8	Implement input validation rules (type, range, format) during the Construction phase of each increment, not after

2.1.3 Contingency Plan

If a risk is not fully avoided despite mitigation efforts, the following fallback actions define how the project will respond to limit damage to the schedule and deliverables.

Risk ID	Contingency Action (if risk occurs)
R1	Reduce scope to core acceptance criteria only (add/edit/delete, reports); defer "nice-to-have" features like advanced filters
R2	Negotiate with supervisor to move the change to a "future enhancement" section of documentation instead of the current build
R3	Roll back to the last stable schema backup and re-migrate data; re-run affected unit tests
R4	Switch to a simpler Tkinter layout (basic widgets only) instead of custom-styled components to save time
R5	Use documented code and commit history to resume quickly; extend working hours temporarily to recover lost time
R6	Restore the latest Git commit from GitHub; rebuild the local environment from the repository
R7	Prioritize testing acceptance criteria (Section 1.4.3) first; treat non-critical bugs as known issues in documentation
R8	Patch validation logic immediately as a hotfix and add a regression test case for the specific input that failed

2.1.4 Risk Monitoring Plan

To ensure risks are caught early rather than discovered after damage occurs, each risk is tracked using a specific monitoring activity at a defined frequency throughout the project.

Risk ID	Monitoring Activity	Frequency
R1	Compare planned vs. actual progress against the WBS/schedule	Weekly
R2	Review requirements and any pending Change Requests with supervisor	Weekly
R3	Verify database schema and run sample-data checks	Every development session
R4	Track time spent per module against the WBS estimate to catch slippage early	Weekly

R5	Self-check progress against milestone dates in Section 1.4.2	Weekly
R6	Check Git repository commit history and backup status	Weekly
R7	Review completed vs. planned test cases	Weekly, and after every increment
R8	Run validation test cases on each new input field	After every module completion

2.2 Project Estimation and Schedule

2.2.1 Estimation Method Used

The Basic COCOMO81 Model (Organic Mode) was selected for effort estimation. Justification:

- The project is small in size (single desktop application, well-understood problem domain), which fits the Organic project profile defined by COCOMO (small team, familiar requirements, not tightly constrained).
- It is developed by a single, small team (1 developer) with a stable, non-rigid requirement set — matching COCOMO's Organic criteria exactly.
- COCOMO is simpler to apply than Function Point Analysis for a solo academic project, since it only requires an estimated code size (KLOC) rather than detailed counts of Internal Logical Files, External Inputs, etc., which are harder to estimate accurately before design is finalized.
- COCOMO gives a quick, defensible, formula-based estimate suitable for early planning, consistent with what was taught for early/rough-order-of-magnitude estimation.

2.2.2 Project Estimation

Estimated Size: Based on the scope (expense/income CRUD, category management, report generation, local SQLite storage, and a Tkinter GUI), the project is estimated at 1.0 KLOC (1,000 lines of source code).

Basic COCOMO81 Formula (Organic Mode):

- Effort: $E = 2.4 \times (\text{KLOC})^{1.05}$
- Development Time: $T = 2.5 \times (E)^{0.38}$

Calculation:

$$E = 2.4 \times (1.0)^{1.05} = 2.4 \text{ Person-Months (PM)}$$

$$T = 2.5 \times (2.4)^{0.38} \approx 3.49 \text{ months } (\approx 14 \text{ weeks})$$

$$\text{Team size} = E / T = 2.4 / 3.49 \approx 1 \text{ developer}$$

Cost Estimation: Assuming a developer cost rate of NPR 30,000/month:

$$\text{Cost} = \text{Effort (PM)} \times \text{Monthly Rate}$$

$$= 2.4 \times 30,000 = \text{NPR } 72,000$$

Note on Schedule Compression: COCOMO's ~14-week estimate assumes typical industrial team overheads like coordination, communication delays, and onboarding. Since this project has a single developer working under direct supervisor guidance with a fixed, narrow scope, these overheads don't apply — allowing the 8-week schedule to be met through focused daily effort rather than by ignoring the estimate.

2.2.3 Project Schedule

The project schedule is presented using a Gantt Chart. It maps all major activities of the Personal Expense Tracker across the 8-week timeline along with the assigned resource (single developer).

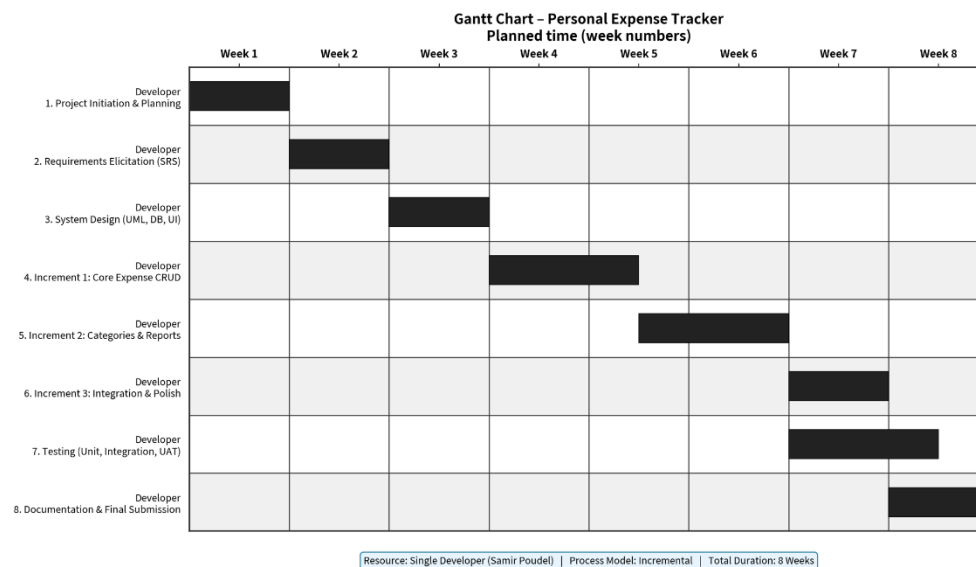


Fig 2.1: Project Schedule Gantt Chart

2.3 Configuration Management Plan

2.3.1 Change Management Plan

Any proposed change to the Personal Expense Tracker is first documented in a Change Request (CR) Log, recording the change description, reason, requester, priority, and expected impact on the schedule. Each change is reviewed against the current project scope

and remaining schedule before a decision is made. Approved changes are incorporated into the next development increment, while rejected requests are archived along with the justification for rejection. All implemented changes are recorded against the software version in which they were introduced, ensuring full traceability throughout the project.

CR ID	Description	Priority	Status	Version Implemented
CR-01	<i>(example)</i> Add yearly report export option	Medium	Approved	v0.8

2.3.2 Version Management Plan

Git is used as the version management tool throughout the project, following a semantic versioning scheme to track the evolution of the software. Each version is committed with a clear, meaningful commit message describing the change, and major development milestones are tagged in the repository for easy reference.

Version	Description
v0.1	Initial prototype
v0.5	Core modules completed
v0.8	Testing version
v1.0	Final project submission

2.3.3 Release Management Plan

Before each release, unit testing and integration testing are carried out to ensure stability. The final release is additionally validated through **User Acceptance Testing (UAT)** prior to submission, confirming that the application meets the acceptance criteria defined in Section 1.4.3.

Release	Description
Release 1	Basic expense recording module
Release 2	Category management and reporting
Release 3	Testing and bug fixes
Final Release (v1.0)	Complete Personal Expense Tracker with documentation

Chapter 3: System Engineering

3.1 Requirements

3.1.1 Domain Requirements

S.N.	Domain Requirement
1	The system must support recording of both income and expense transactions.
2	Every transaction must have a date, amount, category, and optional description.
3	Expense amounts must be positive numeric values; negative or zero amounts are invalid.
4	Categories must be available for classification of expenses (e.g., Food, Transport, Bills, Others).
5	The system must allow calculation of totals for a selected period (daily, monthly, yearly).
6	All financial data must remain private and stored only on the user's local machine.
7	Reports must reflect accurate arithmetic based on recorded transactions.
8	The system must operate offline without requiring an internet connection.

3.1.2 Functional Requirements

Priority	Functional Requirement	Description
High	FR-01: Add Transaction	User can add a new income or expense record with date, category, amount, and description.
High	FR-02: Edit Transaction	User can modify an existing transaction without data loss.
High	FR-03: Delete Transaction	User can delete a selected transaction from the system.
High	FR-04: View Transactions	User can view all recorded transactions in a clear tabular format.
High	FR-05: Category Management	System provides predefined categories and allows custom categories.
High	FR-06: Search and Filter	User can search and filter transactions by date, category, or keyword.

High	FR-07: Generate Reports	System generates daily, monthly, and yearly summary reports with totals.
High	FR-08: Local Data Storage	All data is stored and retrieved from a local SQLite database.
Medium	FR-09: Input Validation	System validates date format, amount range, and required fields before saving.
Medium	FR-10: Category-wise Summary	System displays expense totals grouped by category.

3.1.3 Non-Functional Requirements

S.N.	Category	Non-Functional Requirement
1	Usability	The interface must be simple and intuitive so that a non-technical user can record expenses with minimal training.
2	Reliability	Calculations and data storage must be accurate; the system should not lose data during normal operation.
3	Performance	Report generation and data retrieval must complete within 2–3 seconds for typical personal usage volumes.
4	Maintainability	Source code must be modular and well-structured to allow easy modification of individual modules.
5	Portability	The application must run on Windows 10/11 and Linux without major changes.
6	Security	Data is stored only locally; no external network transmission of financial information is permitted.
7	Availability	The system must be usable offline at any time without dependency on external services.
8	Efficiency	The application should run smoothly on a standard laptop with minimum 4 GB RAM.

3.1.4 Requirement Management Process

The requirement management process for the Personal Expense Tracker followed four main activities: elicitation, analysis, modeling, and specification. This process was selected because the project has a small and stable scope, a single developer, and a fixed 8-week academic timeline. It is consistent with the communication and concept-scoping activities described for defining software product scope.

1. Elicitation: Requirements were obtained through communication with stakeholders. A short questionnaire was prepared and discussed with the project supervisor to identify user needs, expected inputs/outputs, and constraints. Existing personal finance practices (notebooks and spreadsheets) were also reviewed. The problem statement and acceptance criteria from Chapter 1 were used as the primary basis for stakeholder needs. The written description of need was reviewed and refined with the supervisor (similar to a formal technical review of outputs and inputs).

2. Analysis: Elicited requirements were examined and classified into domain, functional, and non-functional categories. Customer-visible inputs and outputs were listed and checked for completeness. Conflicts (for example, advanced features versus the 8-week schedule) were resolved by prioritizing core acceptance criteria and deferring optional features. This ensured that the product scope remained unambiguous and achievable.

3. Modeling Requirements were modeled to define the functionality and behavior of the system:

- **Use Case Model** – shows interactions between the user and the system (*see Annex*)
- **Data Flow Model (DFD)** – shows how data moves through the system processes (*see Annex*)
- **Process Flow Model** – shows the sequence of activities for main operations (*see Annex*)

These models support visual verification of functions and behaviors before construction begins.

4. Specification: All validated requirements were documented in structured form (Sections 3.1.1–3.1.3) and linked to the acceptance criteria. A narrative scope description and the supporting models together form the Software Requirements Specification (SRS). The SRS was frozen after Week 2 to control later requirement changes.

Rationale for Process Selection

- Scope is defined after communication with stakeholders, as recommended for software product scope.
- A questionnaire and supervisor review help avoid misunderstanding of customer needs.
- Modeling of functions/behaviours provides clear work products before coding.
- Freezing the SRS reduces the risk of late requirement changes (Risk R2) and supports the single-developer, time-constrained project.

3.2 Design

3.2.1 System Architecture

The system architecture of the Personal Expense Tracker follows a **layered architecture** style. This architecture divides the application into multiple layers, where each layer has a specific responsibility and only communicates with the layer directly below it.

The layered architecture of the proposed system is shown in the figure below:

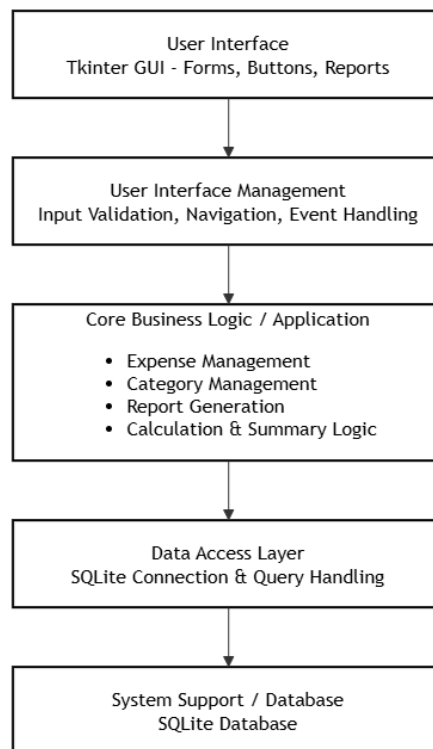


Fig 3.1: Layered Architecture

Description of Layers:

- **User Interface Layer:** Provides the graphical interface using Tkinter. It includes forms, buttons, and report displays for user interaction.
- **User Interface Management Layer:** Handles input validation, navigation between screens, and event handling.
- **Core Business Logic / Application Layer:** Contains the main functionalities of the system such as Expense Management, Category Management, Report Generation, and calculation logic.
- **Data Access Layer:** Manages the connection with the SQLite database and performs all query operations (Insert, Update, Delete, Select).

- **System Support / Database Layer:** The actual SQLite database that permanently stores all expense and category data.

This layered design ensures separation of concerns, easy maintenance, and better modularity of the Personal Expense Tracker application.

3.2.2 High Level Designs

The high-level design of the Personal Expense Tracker follows a modular structure. The system is divided into three major modules: User Interface Module, Business Logic Module, and Data Access Module. Each major module is further decomposed into specific features and interfaces.

The modular structure of the system is shown in the figure below:

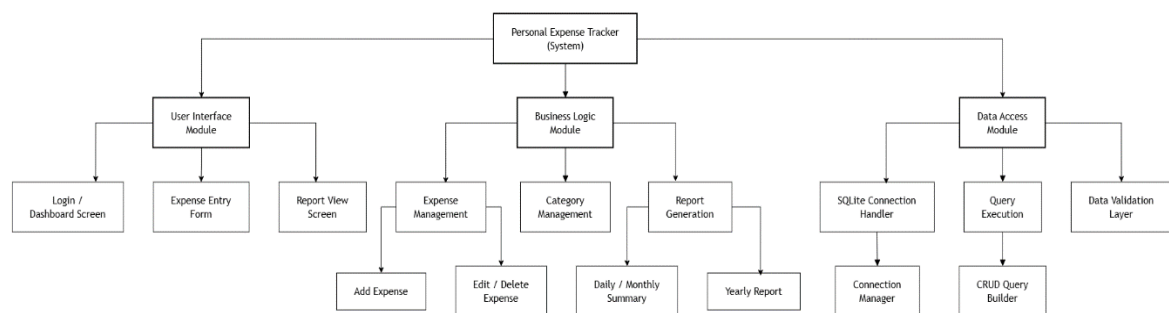


Fig 3.2: High Level Designs

Description:

- **User Interface Module** handles all interaction with the user. It includes the Login/Dashboard Screen, Expense Entry Form, and Report View Screen.
- **Business Logic Module** contains the core functionalities of the system such as Expense Management (further decomposed into Add Expense and Edit/Delete Expense), Category Management, and Report Generation (further decomposed into Daily/Monthly Summary and Yearly Report).
- **Data Access Module** is responsible for database operations including the SQLite Connection Handler (with its Connection Manager), Query Execution (with its CRUD Query Builder), and Data Validation

This modular design ensures separation of concerns, high cohesion, and low coupling. It also supports the Incremental Process Model by allowing each module to be developed and tested independently.

3.2.3 Data Structure Design

The Personal Expense Tracker uses a local SQLite database to store all application data. The data structures are designed around three core entities :User, Category, and Transaction which together support recording, categorizing, and reporting on income and expenses. The entities and their relationships are detailed below.

Entity: User

Stores basic profile information for the person using the application (single local user).

Field Name	Data Type	Description
user_id	INTEGER (PK)	Unique identifier for the user
username	TEXT	Display name shown on the dashboard
created_date	DATE	Date the profile was created

Entity: Category

Stores both predefined (system-provided) and custom (user-created) expense/income categories.

Field Name	Data Type	Description
category_id	INTEGER (PK)	Unique identifier for the category
category_name	TEXT	Name of the category (e.g., Food, Transport, Salary)
category_type	TEXT	Indicates "Income" or "Expense"
is_custom	BOOLEAN	TRUE if user-created, FALSE if predefined

Entity: Transaction

Stores every income and expense record entered by the user. This is the central entity of the system.

Field Name	Data Type	Description
transaction_id	INTEGER (PK)	Unique identifier for the transaction
user_id	INTEGER (FK)	References User.user_id
category_id	INTEGER (FK)	References Category.category_id
amount	DECIMAL	Transaction amount
transaction_type	TEXT	"Income" or "Expense"
date	DATE	Date the transaction occurred
description	TEXT	Optional note about the transaction

Relationships

- One **User** can have many **Transactions** (1-to-many).
- One **Category** can be applied to many **Transactions** (1-to-many).
- Each **Transaction** belongs to exactly one **User** and one **Category**.

Supporting Data Structures

- **Report Summary (derived, not stored):** Daily, monthly, and yearly reports are computed dynamically from the Transaction table using SQL aggregate queries (SUM, GROUP BY), rather than stored as a separate entity — avoiding data duplication and ensuring reports always reflect current data.
- **In-memory structures:** Within the application logic, transaction records are temporarily held as Python objects/dictionaries before being validated and committed to the SQLite database, supporting fast form validation and summary calculations before final storage.

3.3 Testing

3.3.1 Test Justification

Testing is a very important phase of the Personal Expense Tracker project. The main goal of testing is to make sure that the application works correctly, stores data safely, and is easy for users to operate.

1. Why Testing is Required

- To check that all features (Add, Edit, Delete, Reports) work properly.
- To find and fix errors before final submission.
- To ensure data is saved correctly in the SQLite database.
- To confirm that the application is user-friendly.

2. Test Coverage

- All major modules of the system will be tested.
- Both functional features and data validation will be covered.
- Special attention will be given to data persistence (data should remain after closing and reopening the application).

3. Types of Tests and Their Reasons

Type of Test	Reason for Using This Test
Unit Testing	To test small parts of the program (like add expense function) separately.
Integration Testing	To check whether different modules work correctly together.

System Testing	To test the complete application as a whole.
Acceptance Testing	To check whether the system meets the user's requirements.
Regression Testing	To make sure old features still work after adding new features.

4. Testing Process Testing will be done step by step after every increment. First unit testing will be done, then integration testing, and finally system and acceptance testing will be performed before project submission.

3.3.2 Test Plan

1. Introduction:

This test plan describes the testing approach for the Personal Expense Tracker, a Python-based desktop application developed using Tkinter and SQLite.

2. Test Items

- Personal Expense Tracker Application
- User Interface Module
- Business Logic Module
- Data Access Module
- SQLite Database

3. Features to be Tested

- Add, Edit, and Delete Expense
- Category Management
- Report Generation (Daily, Monthly, Yearly)
- Data Validation
- Data Persistence after application restart
- Search and Filter functionality

4. Features not to be Tested

- Multi-user features
- Online / Cloud synchronization
- Advanced security features

5. Approach

- Manual testing for User Interface and overall functionality
- Unit testing for individual functions

- Integration testing between modules
- System testing of the complete application
- Acceptance testing based on defined acceptance criteria

6. Pass / Fail Criteria

- **Pass:** All critical features work correctly and no major defects remain.
- **Fail:** Data loss, incorrect calculations, or application crash in core features.

7. Suspension and Resumption Criteria Testing will be suspended if a critical defect blocks further testing. Testing will resume after the defect is fixed and verified.

8. Test Deliverables

- Test Plan
- Test Cases
- Defect Log
- Test Summary Report

9. Environmental Needs

- Operating System: Windows 10/11 or Linux
- Software: Python 3.x, Tkinter, SQLite
- Hardware: Standard desktop or laptop with minimum 4 GB RAM

10. Responsibilities

- Developer (Student): Prepare test cases, execute tests, and document results
- Supervisor: Review test outcomes

11. Schedule

- Unit Testing: After each increment
- Integration and System Testing: Week 6–7
- Acceptance Testing: Final week

3.3.3 Test Design

Test Design Specification – Personal Expense Tracker

1. Features to be Tested

- Expense Management
- Category Management
- Report Generation
- Database Operations
- User Interface

2. Approach Black-box testing will be used for functional requirements. Limited white-box testing will be applied for critical calculations and validation logic.

3. Test Case Design

Module 1: User Interface Module

Test Case ID	What to Test	Steps to Perform	Expected Result
UI-01	Open Add Expense Form	Click on “Add Expense” button	Add Expense form should open
UI-02	View Report Screen	Click on “View Reports” button	Report screen should open with data

Module 2: Expense Management Module

Test Case ID	What to Test	Steps to Perform	Expected Result
EM-01	Add New Expense	Enter date, amount, category and click Save	Expense should be saved and shown in list
EM-02	Edit Expense	Select an expense and change the amount	Updated amount should be saved
EM-03	Delete Expense	Select an expense and click Delete	Expense should be removed from the list

Module 3: Category Management Module

Test Case ID	What to Test	Steps to Perform	Expected Result
CM-01	Add New Category	Enter category name and click Save	New category should be added
CM-02	Delete Category	Select a category and click Delete	Category should be removed

Module 4: Report Generation Module

Test Case ID	What to Test	Steps to Perform	Expected Result
RG-01	Monthly Report	Select month and click Generate Report	Correct total amount of that month should show

RG-02	Category-wise Report	Select a category and generate report	Expenses of that category should be displayed
-------	----------------------	---------------------------------------	---

Module 5: Database Module

Test Case ID	What to Test	Steps to Perform	Expected Result
DB-01	Data Saving	Add an expense and close the application	Data should still be available after reopening
DB-02	Invalid Amount Check	Enter text or negative number in amount	System should show error message

These test cases cover the main features of the Personal Expense Tracker and help ensure the system works correctly.

Chapter 4: Monitoring & Control

4.1 Monitoring & Control

4.1.1 Time Monitoring

Time monitoring of the Personal Expense Tracker project was carried out using a Gantt Chart. The Gantt Chart presented the planned schedule of all major activities across the 8-week timeline and was used to track actual progress against the plan on a weekly basis.

The Gantt Chart used for time monitoring is the same as shown in Figure 2.1 (Chapter 2). Throughout the project, progress was reviewed every week. All major milestones (Requirements, Design, Implementation, Testing, and Final Submission) were completed according to the planned schedule. The project was finished within the allocated 8 weeks without any major delay.

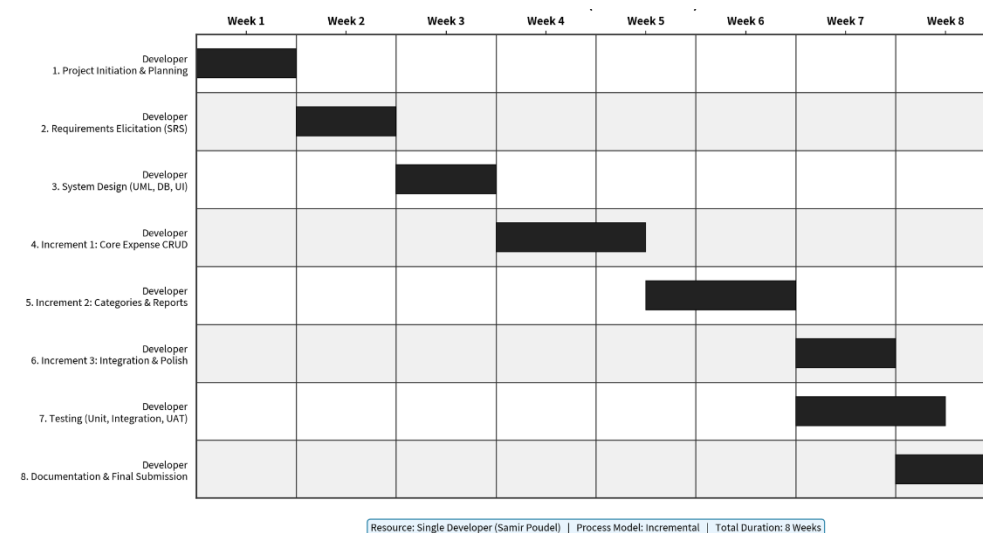


Fig 4.1: Gantt Chart after completion of project

4.1.2 Cost Monitoring

Cost monitoring of the Personal Expense Tracker project was carried out using the Earned Value Management (EVM) technique.

Assumptions:

Total planned effort = 40 person-days (8 weeks × 5 working days)

Developer worked according to the planned schedule

Earned Value Parameters (at project completion):

Parameter	Value	Meaning
PV	40 person-days	Total planned work
EV	40 person-days	Work actually completed

AC	40 person-days	Actual effort spent
SV	0	No schedule variance (project on time)
CV	0	No cost/effort variance
SPI	1.0	Project completed exactly on schedule
CPI	1.0	Effort used matched the planned effort

Interpretation:

SPI = 1.0 means the project finished exactly on time.

CPI = 1.0 means no extra effort was required.

Both indicators show that the project was successfully monitored and controlled.

4.1.3 Control Measures

The following table shows possible deviations and the control measures that will be taken:

Type of Deviation	Possible Causes	Control Measures (Actions to be Taken)
Time Delay	Module taking more time than planned	Re-plan remaining tasks, work extra hours, reduce scope of less important features
Time Delay	Technical problems / bugs	Prioritize critical bugs, take help from supervisor
Cost Overrun	Spending more effort/time than planned	Focus only on core features, avoid unnecessary enhancements
Both Time & Cost Deviation	Poor estimation or changing requirements	Review the plan, freeze requirements, follow Incremental approach strictly
Schedule Behind	Delayed testing or documentation	Parallel work on documentation while testing

These control measures will help keep the project on track in terms of both time and cost.

Project Outcome:

During the development of the Personal Expense Tracker, no major time or cost deviation occurred. Regular monitoring using the Gantt Chart and Earned Value parameters helped keep the project on track. As a result, the project was completed successfully within the planned 8-week schedule.

4.2 Quality Assurance

4.2.1 QMS Selection

The Quality Management System selected as the base reference for the Personal Expense Tracker project is **ISO 9001**.

Reasons for selecting ISO 9001:

- It provides a clear framework for defining processes, reviewing work products, and controlling quality.
- It is suitable for small projects because its core principles (plan, do, check, act) can be applied in a lightweight way.
- It supports process compliance, reviews, and continuous improvement — all of which fit the Incremental development approach used in this project.
- It is internationally recognized and aligns with the quality practices taught in Software Engineering.

A full formal ISO certification is not required for this academic project; only the relevant process and quality practices of ISO 9001 are followed.

4.2.2 Quality Factors

The quality factors considered for the Personal Expense Tracker, listed in order of preference, are:

Preference Order	Quality Factor	Reason for Preference
1	Correctness	Calculations and stored data must be accurate
2	Reliability	Application must not crash or lose data during normal use
3	Usability	Interface must be simple for non-technical users
4	Maintainability	Code must be modular so future changes are easy
5	Efficiency	Reports and data operations must respond quickly
6	Portability	Application should run on Windows and Linux

4.2.3 Metrics for Process and Process Measurements

The following metrics were selected to measure process and product quality of the Personal Expense Tracker:

1. Process Metrics

Metric	Description	Calculation / Result
Schedule Variance	Difference between planned and actual dates	Planned = 8 weeks Actual = 8 weeks SV = 0 (On time)
Effort Variance	Planned effort vs actual effort	Planned = 40 person-days Actual = 40 person-days Effort Variance = 0
Defects found per increment	Defects detected after each increment	Increment 1: 3 defects Increment 2: 2 defects Increment 3: 1 defect Total = 6 defects
Defect Removal Efficiency (DRE)	$DRE = E / (E + D)$	E = 6 (defects found before delivery) D = 0 (defects found after delivery) $DRE = 6 / (6+0) = 1.0$ (100%)

2. Product Metrics

Metric	Description	Calculation / Result
Size (KLOC)	Approximate size of source code	≈ 1.0 KLOC
Defects per KLOC	Defects found / Size	6 defects / 1.0 KLOC = 6 defects/KLOC
Test Cases Passed	Percentage of test cases passed	12 test cases planned 12 passed 100%
Requirements Coverage	% of functional requirements implemented	10 functional requirements 10 implemented 100%

Where These Metrics Were Implemented

Metric	Where Applied in the Project
Schedule Variance	Compared weekly with Gantt Chart (Chapter 2 & 4.1.1)
Effort Variance	Compared with planned effort in Cost Monitoring (4.1.2)
Defects found per increment	Recorded during unit and integration testing of each increment

Defect Removal Efficiency (DRE)	Calculated at the end of testing phase
Size (KLOC)	Estimated during project estimation (Chapter 2)
Defects per KLOC	Calculated after final testing
Test Cases Passed	Measured from Test Design results (Section 3.3.3)
Requirements Coverage	Checked against Functional Requirements (Section 3.1.2)

These metrics helped confirm that the process was controlled and the final product met the required quality level.

These metrics are simple, suitable for a single-developer academic project, and provide clear visibility of process performance and product quality.

4.2.4 Quality Monitoring

Quality of the Personal Expense Tracker was monitored, measured, and controlled through the following activities:

1. Monitoring Activities

- Weekly progress review against the Gantt Chart and milestones
- Code and design reviews after each increment
- Unit testing after development of each module
- Integration testing after combining modules
- System and Acceptance testing before final submission
- Tracking of defects found and fixed in each increment

2. Templates / Records Used

- Test Case Table (Module-wise test cases)
- Defect Log (list of defects, severity, and status)
- Change Request Log
- Weekly progress notes
- Test Summary Report

3. References against which Quality was Measured

- Acceptance Criteria defined in Section 1.4.3
- Functional and Non-Functional Requirements (Section 3.1)
- Test Plan and Test Design (Section 3.3)
- Planned schedule and effort estimates (Chapter 2)

- Selected Quality Factors (Correctness, Reliability, Usability, etc.)

4. Control Actions

- Defects found during testing were fixed before moving to the next increment
- Any schedule slippage was controlled by focusing only on core features
- Requirements were frozen after Week 2 to avoid late changes
- Final quality was verified through successful execution of all critical test cases and acceptance criteria

Through these activities, the quality of both the development process and the final product was continuously monitored and controlled throughout the project.

Annex 1: Requirement Models

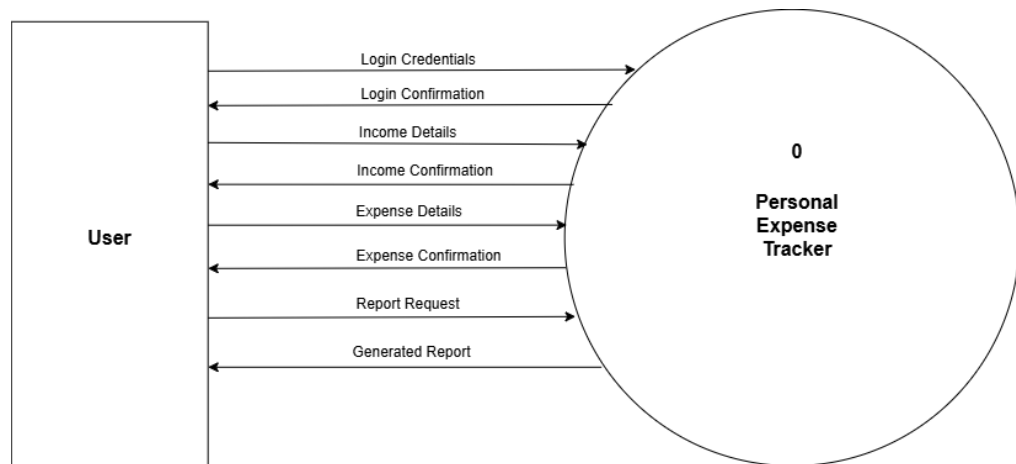


Fig 5.1: DFD Level -0

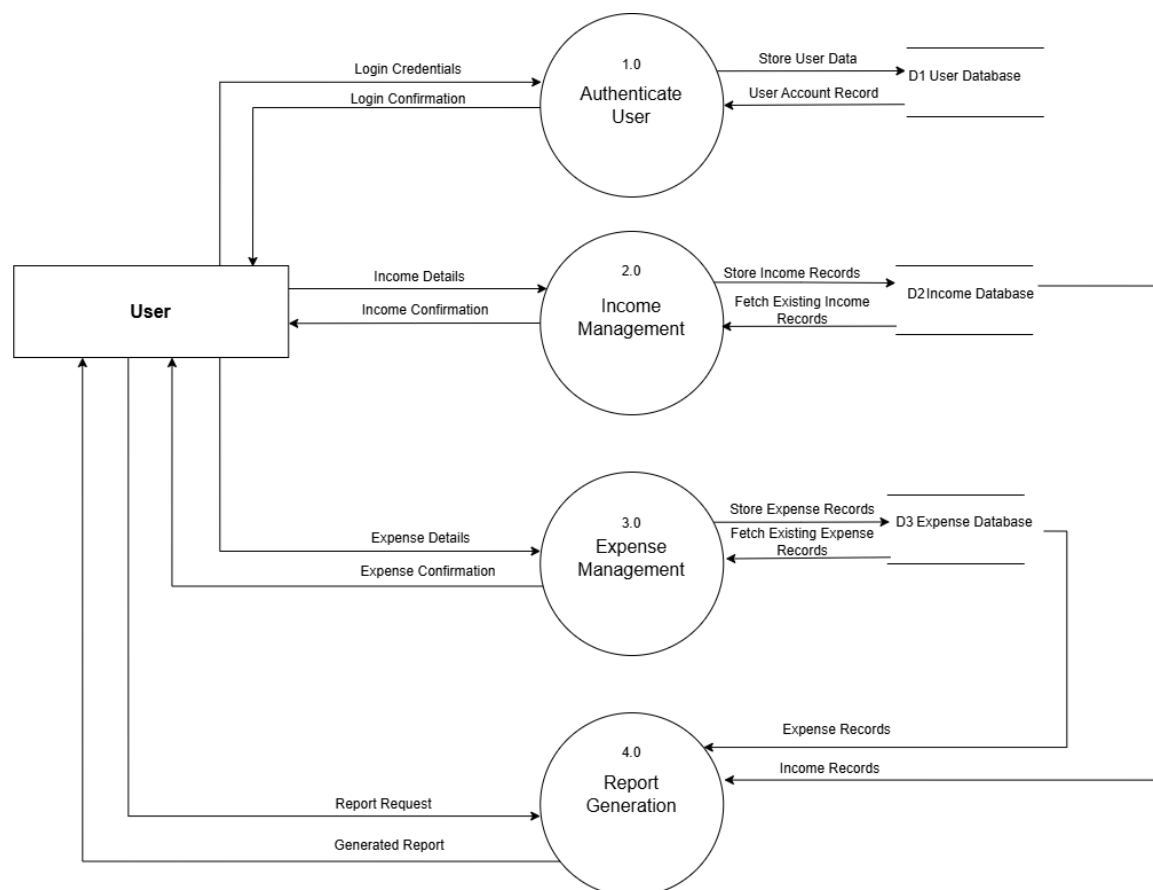


Fig 5.2: DFD Level -1

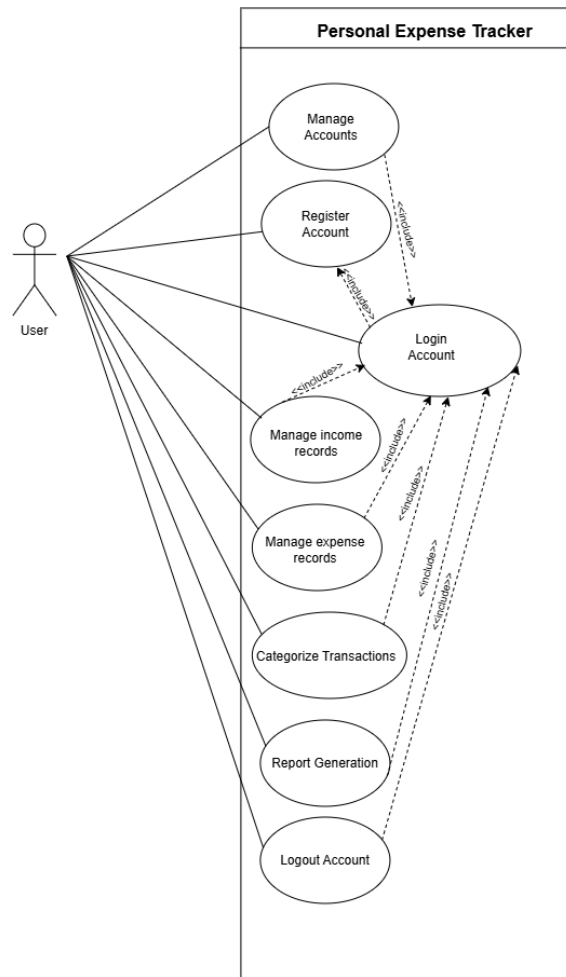


Fig 5.3: Use Case Diagram

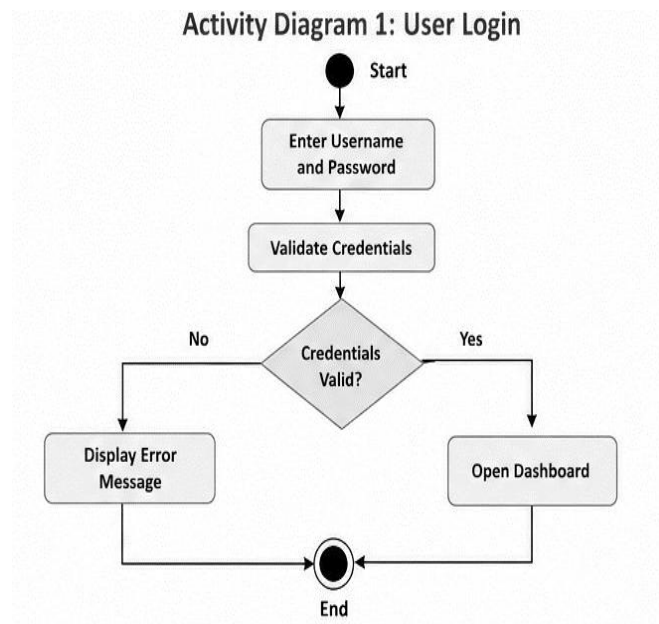


Fig 5.4: Activity Diagram-1(Login module)

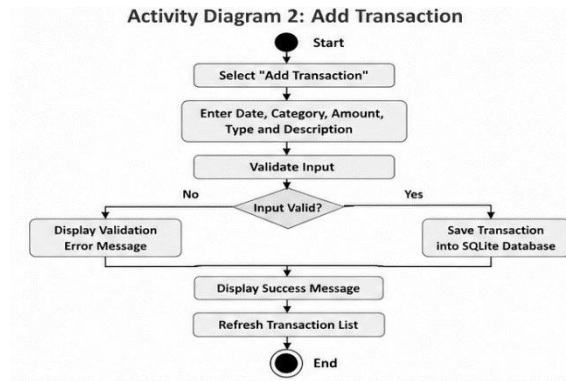


Fig 5.5: Activity Diagram-2(Add Transaction Module)

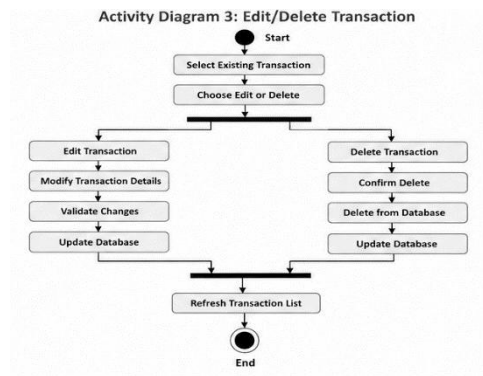


Fig 5.6: Activity Diagram-3(Manage Transaction Module)

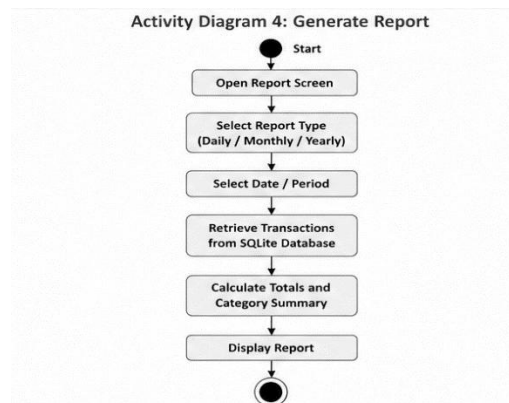


Fig 5.7: Activity Diagram-4(Generate Report Module)